

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522- Ağ Tabanlı Paralel Dağıtım Sistemleri

PROJE 3 RAPORU

Veritabanı Yedekleme ve Otomasyon Çalışması

Emine Bülbül

21290189

06 Haziran 2026

<https://github.com/eminebulbul/FinalHospitalDB>

İÇİNDEKİLER

1. PROJENİN AMACI VE KAPSAMI	3
2. KULLANILAN TEKNOLOJİLER VE SİSTEM MİMARISI	3
3. OTOMASYON VE DENETİM MİMARISI	3
3.1. DİNAMİK YEDEKLEME MEKANİZMASI.....	4
3.2. DENETİM GÜNLÜĞÜ VE HATA KONTROLÜ	4
3.3. YÖNETİCİ UYARILARI VE SİSTEM BİLDİRİMLERİ (PUSH NOTIFICATIONS)	5
4. ZAMANLAYICI KONFIGÜRASYONU VE GÖREV YÜRÜTME (CRON JOB)	5
5. SONUÇ VE KAZANIMLAR	5

1. Projenin Amacı ve Kapsamı

Kritik veri tabanı sistemlerinde yedekleme işlemlerinin manuel olarak yapılması, insan hatası (Human Error) ve unutulma riskini beraberinde getirir. Endüstri standartları, yedekleme süreçlerinin tamamen otomatize edilmesini, her işlemin denetlenmesini ve olası hatalarda sistem yöneticilerinin anında uyarılmasını şart koşmaktadır. Veri kaybı senaryolarında kurtarma süresinin (Recovery Time Objective - RTO) minimuma indirilmesi için sistemin kendi kendini idame ettirebilmesi zorunludur.

Bu projenin amacı, MS SQL Server Agent gibi aracı yazılımlara bağımlı kalmadan, doğrudan Unix (macOS) işletim sistemi çekirdeği seviyesinde tam teşekküllü bir veritabanı otomasyonu kurmaktır. Çalışma kapsamında; Bash Scripting ile akıllı bir yedekleme programı yazılmış, loglama (raporlama) mekanizması kurulmuş ve sistem API'leri kullanılarak anlık hata/başarı bildirimleri tasarlanmıştır.

2. Kullanılan Teknolojiler ve Sistem Mimarisi

Proje, veritabanı yönetimini işletim sistemi otomasyonu ile entegre edecek şekilde geliştirilmiştir.

- **Veritabanı Motoru:** PostgreSQL 15+ (pg_dump aracı)
- **Otomasyon Dili:** Bash Scripting (.sh betikleri)
- **Zamanlayıcı Servisi:** Unix Cron Daemon (crontab)
- **Bildirim (Notification) API:** AppleScript (osascript)
- **Çalışma Ortamı:** macOS Terminal (Zsh/Bash)

3. Otomasyon ve Denetim Mimarisi

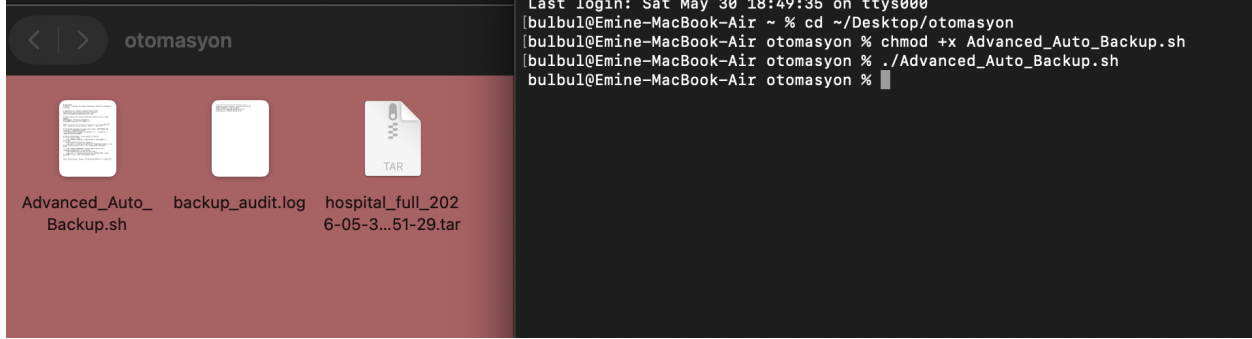
Otomasyon sürecinin "beyni" olarak Advanced_Auto_Backup.sh adında özel bir betik (script) tasarlanmış ve sisteme terminal üzerinden "Executable" (Çalıştırılabilir) yetkisi verilerek bağımsız bir arka plan uygulaması haline getirilmiştir.

```
proje 3 > otomasyon > $ Advanced_Auto_Backup.sh
1  #!/bin/bash
2  # PROJE 7: Gelişmiş Veritabanı Yedekleme, Denetim ve Otomasyon Çalışması
3
4  # Yedeklerin ve logların kaydedileceği klasör
5  BACKUP_DIR="/Users/bulbul/Desktop/4.sınıf2.dönem/ag/projeler/proje 3/otomasyon"
6  LOG_FILE="$BACKUP_DIR/backup_audit.log"
7
8  # Dosya isimlerinin sonuna eklenecek dinamik tarih ve saat damgası
9  DATE=$(date +%Y-%m-%d_%H-%M-%S)
10 FILE_NAME="hospital_full_$DATE.tar"
11
12 echo "-----" >> $LOG_FILE
13 echo "Yedekleme İşlemi Başladı: $DATE" >> $LOG_FILE
14
15 # PostgreSQL pg_dump ile yedek alma işlemi
16 /opt/homebrew/bin/pg_dump -U postgres -F c -d hospital -f "$BACKUP_DIR/$FILE_NAME"
17
18 # Denetim Mekanizması: İşlem başarılı oldu mu?
19 if [ $? -eq 0 ]; then
20     echo "DURUM: BAŞARILI. Yedek Dosyası: $FILE_NAME" >> $LOG_FILE
21 else
22     echo "DURUM: BAŞARISIZ! Sistem yöneticisinin acil müdahalesi gerekiyor." >> $LOG_FILE
23 fi
24
25 echo "İşlem Bitişi: $(date +%Y-%m-%d_%H-%M-%S)" >> $LOG_FILE
```

Şekil 1. Bash Script

3.1. Dinamik Yedekleme Mekanizması

- **Teorik Altyapı:** Aynı isimle alınan yedekler, eski yedeklerin üzerine yazılarak geçmişe dönük kurtarma senaryolarını imkansız hale getirir. Bu nedenle her yedek dosyasının benzersiz (unique) olması gerekir.
- **Teknik Uygulama:** Bash script içerisinde `$(date +"%Y-%m-%d_%H-%M-%S")` değişkeni kullanılarak sisteme dinamik tarih ve saat damgası ürettirilmiştir. Böylece `pg_dump` aracı her tetiklendiğinde `hospital_full_2026-06-02_03-00-00.tar` formatında, üzerine yazılmayan ve kronolojik olarak sıralanabilen yedek dosyaları oluşturulmuştur.

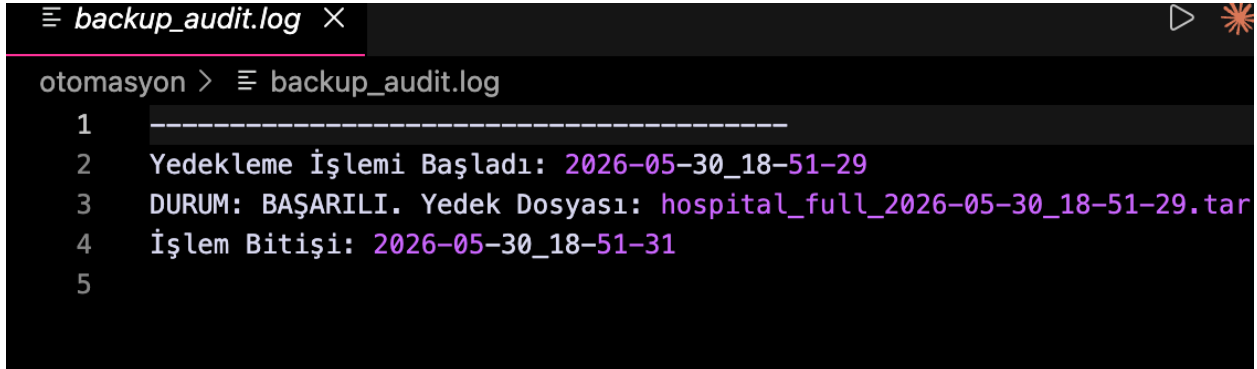


Şekil 1. Dinamik Tarih Damgalı Yedekleme İşlemi

3.2. Denetim Günlüğü ve Hata Kontrolü

- **Teorik Altyapı:** Otomatize sistemlerde (özellikle gece saatlerinde çalışan görevlerde) işlemlerin arka planda başarılı olup olmadığını kanıtlamak için düzenli denetim (Audit) raporları üretilmesi gerekmektedir (PowerShell/T-SQL scripting raporlama alternatifi).

Teknik Uygulama: Script içerisine bir IF-ELSE koşullu kontrol bloğu (Exit Status Check - `$?`) entegre edilmiştir. Unix mimarisinde `pg_dump` işlemi bittiğinde, sistem işletim sistemine bir çıkış kodu döner. Eğer dönen kod '0' ise işlem hatasız tamamlanmış demektir. İşlemin başlangıç saati, bitiş saati ve başarı durumu saniyesi saniyesine `backup_audit.log` adlı metin dosyasına `append (>>)` yöntemiyle yazdırılarak tam bir geriye dönük raporlama mekanizması sağlanmıştır.



Şekil 2. Veritabanı Denetim ve Raporlama (Audit Log) Dosyası

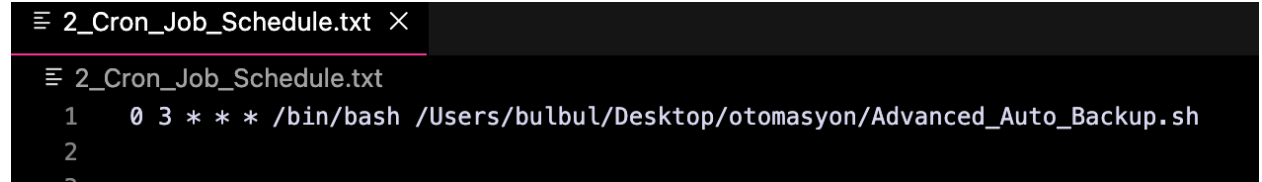
3.3. Yönetici Uyarıları ve Sistem Bildirimleri (Push Notifications)

Teorik Altyapı: Log dosyaları kalıcı kayıtlar sağlasa da, kritik sunucularda disk dolması veya bağlantı kopması gibi acil durumlarda sistem yöneticilerine (DBA) anlık bildirim (Alert) gönderilmesi operasyonel devamlılık için şarttır.

Teknik Uygulama: Bash script içerisindeki mantıksal kontrol bloğuna, macOS sisteminin yerleşik osascript API'si entegre edilmiştir. Script bir hata tespit ettiğinde işletim sistemi ekranın sağ üst köşesinden "KRİTİK HATA: Yedek alınamadı!" şeklinde bir uyarı fırlatacak; işlem başarılı olduğunda ise yöneticinin ekranına "Yedekleme başarılı" bildirimini iletecek şekilde konfigüre edilmiştir.

4. Zamanlayıcı Konfigürasyonu ve Görev Yürütme (Cron Job)

- **Teorik Altyapı:** Hazırlanan bu akıllı scriptin, insan müdahalesine gerek kalmadan periyodik olarak çalıştırılması gerekmektedir (SQL Server Agent alternatifi).
- **Teknik Uygulama:** Unix sistemlerinin görev zamanlayıcısı olan crontab düzenleyicisine müdahale edilerek, otomasyon programı sistemin çekirdeğine işlenmiştir.



```
2_Cron_Job_Schedule.txt X
2_Cron_Job_Schedule.txt
1 0 3 * * * /bin/bash /Users/bulbul/Desktop/otomasyon/Advanced_Auto_Backup.sh
2
```

Şekil 4. Yukarıdaki parametre ile sistemin; yılın her ayı, her günü, saat tam 03:00'te bu scripti arka planda otomatik olarak tetiklemesi garanti altına alınmıştır.

5. Sonuç ve Kazanımlar

Bu proje ile veritabanı yedekleme süreçleri manuel bir iş yükü olmaktan çıkarılarak tam otonom bir sisteme dönüştürülmüştür. Üçüncü parti yazılımlar yerine işletim sisteminin native (doğal) araçları (Bash ve Cron) kullanılarak güvenlik ve performans maksimize edilmiştir. Dinamik dosya isimlendirme ile veri ezilmesi engellenmiş, Audit Log yapısı ile geriye dönük denetilebilirlik sağlanmış ve sistem API bildirimleri ile hata tespiti anlık hale getirilerek kurumsal standartlarda bir altyapı sunulmuştur.